Field Agent App — Build Plan

> **Operative plan for `apps/agent`.** Written 2026-08-16, after assessing `docs/CLBIPP_FieldAgentWireframes_V2.html` against the three HR documents and against what is actually built in this repo.
>
> Budget: **one week.** The customer app took several weeks; this one cannot. The reason it's possible is that the foundation — monorepo, schema, auth, component library, PDF pipeline, smoke harness, seed — already exists and is proven. This plan is written to spend the week on *screens and flows*, not on re-deriving infrastructure.
>
> Read `PLAN_V2_CUSTOMER_APP.md` for the equivalent document for the vendor app; this one follows the same shape deliberately.

---

0. Wireframe assessment — summary

`CLBIPP_FieldAgentWireframes_V2.html` (21 screens, B01–B07) is **good and mostly reusable**. Its layouts, design system and happy path are sound and match the vendor app. **We are not rebuilding it.** But it has nine substantive problems, and every one of them changes *what gets built*, not just how.

What it gets right (keep as-is)

- Same design system as the vendor app — chips, banners, `pickupRow`, timeline, card rhythm all map onto `@clbipp/ui` components that already exist.
- The bulk hub drop-off trio (`dropoff-select` → `dropoff-confirm` → `dropoff-receipt`) is the strongest part of the document and maps directly to build-doc §3.2 and §5 step 5. Build it close to the drawing.
- Job list, on-site assessment, photo proof, wallet/earnings, lifecycle watch-only after drop-off — all correct per the HR docs.
- The "agent sees all, vendor sees little" mirror rule is right and is exactly the inverse of the rule already enforced in the customer app.

What it gets wrong (found 2026-08-16)

#	PROBLEM	CONSEQUENCE
W1	**No safety-checklist screen anywhere.** All three HR docs call it *mandatory* and *pre-pickup*; `SafetyChecklist` already exists in the schema, unused.	The one feature that makes this a *battery* app is missing. **Added as Batch 2.**
W2	**Intake only works for li-ion.** Asks SoH %, kWh, cycle count, BMS temp — a lead-acid battery has none of these. Yet HR §7 says do **portable + automotive lead-acid first**.	The assess→quote spine covers 1 of 4 categories. **Resolved by D0 + D1.**
W3	**Single-battery assumption throughout.** Our model is `Pickup` → `BatteryItem[]`; a fleet pickup with six packs has no path.	**Resolved by D1** — full per-item loop.
W4	**Invents a parallel status vocabulary.** `Draft` / `Quoted` / `Pending drop-off` chips, and a 6-stage timeline (Collected → Transit → Warehouse → Refurb/Recycle → QA → Done) that maps to nothing in `LIFECYCLE_STAGES`. Never uses `arrived`, which Batch 7A added for exactly this app.	Would force a screen to re-declare the stage list, which `CLAUDE.md` forbids. **Resolved by D5.**

#	PROBLEM	CONSEQUENCE
W5	**The ₹140 per-job pickup fee is invented**, not in any HR doc, and has no column or rule.	**Resolved by D3** — kept, with a real rule and a real column.
W6	**Chat + VoIP call are two screens of paid infrastructure.** HR lists in-app chat/call under the *Customer* app. Masked VoIP needs Twilio/Exotel — a paid Indian number and KYC.	**Cut (D4).**
W7	**Login is "Agent ID + OTP".** No backing; phone SMS OTP was rejected in Plan v2 (paid provider + DLT registration).	**Cut** — email + password, same as the customer app.
W8	**Turn-by-turn navigation** needs a routing SDK; the wireframe's own map screen says Leaflet + OSM, which doesn't do turn-by-turn.	**Cut (D4)** — static map + Google Maps deep link.
W9	**The cross-app seam is undefined.** The `pickup` screen just assumes "Vendor accepted the estimated offer". Who writes `offered`? Who writes `collected`? Two apps, one state machine.	**Highest integration risk. Resolved by D7.**

Smaller, folded into the batches below: "Escalate to admin" goes nowhere · no agent onboarding or safety-training surface despite `safetyTrainedAt` in the schema · Cash out / Notifications / "Weekly incentive bonus" are buttons with no rule or destination · `history` rows link to themselves · hub drop-off names a receiving staff member with no hub-staff surface · no completed-job detail view.

---

1. Decisions (D0–D10)

D1–D4 were answered by Aamir on 2026-08-16; **D0 on 2026-08-18**. D5–D10 are carried as stated assumptions — flagged here rather than asked, to keep this to one approval gate. **Do not re-litigate these mid-build.**

D0 — The decision engine is live code, not a fixed artifact ✅ decided

`packages/decision-engine` is an early-days build. **Its logic is the asset; its code is not frozen.** It may be corrected, extended or refactored where that is the right answer. Two consequences:

- **Where the engine and the HR documents disagree, the HR documents win.** The engine was written before the company's flow document arrived; it is not a specification.
- The two defects in §D9 get **fixed inside the engine**, not worked around.

This replaces the "PARKED — do not edit" status the engine carried through the vendor sprint. That status was about *scope* (it wasn't the vendor app's problem), not about the code being sacred. It is this app's problem now.

⚠ It still has **20 passing tests**. Changes come with tests, and a change that alters a pricing output must say so explicitly in its PR — silent economics drift is the one failure mode here that nobody would notice until a demo.

D1 — All four battery categories; the engine runs on li-ion only ✅ decided

The agent works **item by item** through `BatteryItem[]`. Per item they confirm category, chemistry, weight, condition and photos. Then:

- **Li-ion items** (`li_ion_nmc` / `li_ion_lfp` / `li_ion_nca`) → full path: damage rubric → decision engine → pathway + price band.
- **Everything else** (lead-acid, NiMH, other) → **simple path**: weight × condition, priced off `PricingRate` via the existing `estimateQuote` in `packages/core/src/booking.ts`. No engine, no rubric, no pathway.

This is the truest reading of the HR docs and the only option consistent with what the customer app already books. It is also the single biggest cost in this plan (~1.5 extra days vs. a li-ion-only build) — it is why Batch 3 exists as its own batch.

We still do not extend the engine to lead-acid — but for a better reason than "the package is parked" (D0 removed that). Lead-acid is a **mature commodity**: it is bought and sold at a published ₹/kg, and there is no reuse-vs-refurbish-vs-recycle decision to make — essentially all of it is recycled. Running it through a pathway-selection engine would be machinery around a question that has one answer. `PricingRate` models it correctly and already exists.

If Batch 4 finishes early, extending the engine's `Chemistry` set is now *permitted*. It is not planned, and it is not on the critical path.

D2 — Jobs are pushed, not pulled ✅ decided

`Pickup.agentId` is set when the pickup is scheduled. The agent's home lists **only their own assigned jobs**. There is no nearby-jobs feed, no accept-from-a-pool, no distance ranking, no claim race.

- The wireframe's `request-detail` screen survives as an **assigned-job detail** screen (same layout, no Decline-into-the-pool semantics).
- Declining becomes "unable to take this job" → unassigns and flags for admin.
- Saves a hard RLS problem (an agent SELECTing pickups they don't own) and roughly a day.

D3 — The per-job agent fee stays, with a flat rule ✅ decided

A visible pickup fee, credited to the agent's wallet on collection.

- **Rule (v1):** `base_fee + per_km_rate × distance_km`, rounded to the rupee, computed in `packages/core/src/agent-fee.ts` from constants in that one file. Not admin-configurable this sprint (there is no admin app yet).
- **Schema:** one nullable `agent_fee_paise` on `Pickup`.
- **Ledger:** a `WalletTxn` on the agent's profile at collection. `WalletTxnKind` gains `agent_fee`.
- Integer paise, like all money in this repo. Never a float.

This keeps `request-detail`, `navigate`, `pickup`, `receipt` and the whole profile/earnings screen honest — without it those screens have almost no content.

D4 — Chat, VoIP call and turn-by-turn are cut ✅ decided

- **Call** → `tel:` link on the job detail screen.
- **Directions** → static Leaflet + OpenStreetMap map (already the wireframe's own choice for the map screen) + an "**Open in Google Maps**" deep link built from `Address.lat/lng`, which already exist.
- **Chat** → deleted. Not built, not stubbed.

Deletes 3 screens and all associated infrastructure. Same call we made on SMS OTP and Apple sign-in, for the same reason.

D5 — The locked nine-stage lifecycle is not touched

The wireframe's vocabulary maps onto the existing contract; it does not replace it. **No migration to `PickupStatus`.**

WIREFRAME CHIP	REAL STATE
`Draft`	A pickup at `arrived` whose items aren't all confirmed yet. Not a status — a derived UI state.

WIREFRAME CHIP	REAL STATE
`Quoted`	`offered`
`Pending drop-off`	`collected`, with the pickup not yet on a `DispatchManifest`. Derived, not a status.
Timeline "Transit / Warehouse / Refurb / QA / Done"	The real stages: `collected → tested → processed → recovered → certified`, rendered through **`lifecycle-view.tsx`** from `@clbipp/ui`.

No screen re-declares the stage array. Use `isLifecycleStage` / `isStageBefore` / `STAGE_LABELS` from `@clbipp/ui`, per `CLAUDE.md`. `arrived` finally gets its intended use: the agent taps "Arrived" on site.

D6 — Agents do not self-sign-up

Agent accounts are created by seed (and later by the admin app). The agent app ships **login only** — no signup, no account-type selector, no onboarding screen. `role: 'agent'` is set at creation and `authenticated` has no write privilege on `profiles.role` (`supabase/grants.sql`), so this is also the secure default. `safetyTrainedAt` is displayed on the profile screen, read-only.

D7 — The cross-app seam: who writes what

This is the highest-risk part of the build. The contract:

TRANSITION	WRITTEN BY	WHERE
`requested → scheduled`	admin/seed	(out of scope this sprint)
`scheduled → arrived`	**agent**	Agent app, "Arrived" on job detail
`arrived → offered`	**agent**	Agent app, "Present offer" — creates the `Offer` row
vendor accepts	**vendor**	Customer app `/offer` — sets `Offer.acceptedAt`, **status stays `offered`**
`offered → collected`	**agent**	Agent app, "Confirm collection"
`collected → …`	admin/seed	(out of scope)

This changes the customer app. `acceptOffer` in `apps/customer/src/app/(app)/handover/actions.ts` currently jumps `offered → collected` from the *vendor's* side. That was correct when no agent app existed; it is wrong now — the vendor cannot mark their own battery collected. The fix is small and additive:

- New nullable `Offer.acceptedAt`.
- `acceptOffer` sets `acceptedAt` and leaves `status` at `offered`.
- The agent's Confirm-collection action is the only writer of `collected`.
- ⚠ **It also fixes the known `/handover`-mutates-on-GET bug** (flagged in Batch 6.5, still open) — that action becomes a POST as part of this change.

All agent lifecycle writes go through **service-role server actions** that re-verify `pickup.agentId === session.user.id` in code, exactly the pattern `handover/actions.ts` established. Agents get **no** UPDATE policy on `pickups`.

D8 — Money units at the engine boundary

The decision engine returns **rupee floats** (`net_value: number`, `p_min`…). Everything in this repo is **integer paise**.

- One conversion helper, `rupeesToPaise()`, in `packages/core`. Round **half-up at the paise level** on the way in; never round-trip back to a float.
- `Offer.estimatedPrice` (paise) is set from `pricing.p_recommended`.
- `PathwayDecision` keeps the engine's `Decimal(12,2)` rupee values verbatim as the audit record — it is the engine's own log, not a money surface.
- Every ₹ on screen goes through `formatPaise` from `@clbipp/core/format`.

D9 — Market data for the engine, and two traps in it

The engine requires a `MarketData` bundle. The `MarketPrices` table exists and is empty.

- Seed one row in `reset-demo.ts`; read the latest row and map it into `MarketData` in a single adapter in `packages/core`.

Two real defects, both **fixed in the engine** under D0:

- ⚠ **Defect 1 — `StaleMarketDataError` at >24h** (`layers/intake.ts`, `MARKET_FRESHNESS_MAX_HOURS = 24`). A row seeded on Monday **breaks the demo on Tuesday**. Freshness is correct behaviour for live trading and wrong for a seeded demo, so make the window a `Config` field rather than a module constant, and have the adapter stamp `snapshot_timestamp` at read time in `simulated` mode (same posture as `PAYMENTS_MODE`). Do not discover this during the demo.
- ⚠ **Defect 2 — `trace_id` is a module-level in-memory counter** (`layers/intake.ts`, `traceCounter`). On Vercel every cold start resets it, so trace IDs **will collide across requests** — which quietly corrupts the audit trail the whole "Why" screen is built on. Accept a caller-supplied `trace_id` and derive it from the pickup/item id; keep the counter only as a test fallback.

D10 — The agent app is read-scoped by Prisma, not by new RLS

`supabase/policies.sql` today is entirely vendor-scoped; there are **no agent SELECT policies**. Rather than write a whole new policy layer under time pressure, the agent app follows the customer app's proven pattern:

- **Reads** — Prisma in server components, scoped by `agentId` **in code** (Prisma bypasses RLS by design).
- **Writes** — service-role server actions that re-verify ownership in code.
- **Storage** — agent photos go up through a **server** action using the service role. `storage-policies.sql` already anticipates this: "*The agent's on-site photos are written by the service role.*" No new storage policy needed.
- **Realtime** — the agent's watch-only timeline subscribes to `status_events`, which is already in the publication. ⚠ RLS on `status_events` is vendor-scoped, so an **agent's browser subscription will silently receive nothing**. Either add one agent-scoped SELECT policy on `status_events`, or poll on that one screen. Decided in Batch 8; the policy is ~6 lines and is the better answer.

Any new SQL lands in a versioned file under `supabase/`, never only in the dashboard.

---

2. Corrected screen map

19 screens. Wireframe had 21: 3 cut, 4 added, 14 kept (most with changes).

A. Entry & day view

SCREEN	ROUTE	CHANGE FROM WIREFRAME
Login	`/login`	**Email + password**, not Agent ID + OTP (W7). Role-gated to `agent`.
Day view	`/`	Keep. Drop "New requests nearby" (D2). Stats become **Assigned today / Collected today / Earned today** — drop "Avg margin" from home; it's a business figure, odd on a contractor's home screen. Keep the offline banner and resumable-draft row.
Day view — empty	`/` (branch)	Keep as-is.

B. Job → arrival

SCREEN	ROUTE	CHANGE
Job detail	`/job/[id]`	Was `request-detail`. Assigned job, not a pool offer (D2). Shows vendor, address, category, declared items, agent fee. Actions: **Open in Google Maps**, **Call** (`tel:`), **Arrived**.
~~Navigate~~	—	**Cut** (D4). Folded into job detail.
~~Chat~~ / ~~Call~~	—	**Cut** (D4).
Safety checklist	`/job/[id]/safety`	**NEW (W1).** Mandatory gate between `arrived` and intake. Chemistry-aware items: terminals insulated · no puncturing · fire-safe crate · no mixed chemistry · PPE. Writes `SafetyChecklist`. **Cannot proceed until `passed`.**

C. Intake & assessment

SCREEN	ROUTE	CHANGE
Item list	`/job/[id]/items`	**NEW (W3/D1).** The spine of the multi-item flow: every `BatteryItem` on the pickup with a confirmed/pending state, and a running total. This is what the wireframe was missing.
Item confirm	`/job/[id]/items/[itemId]`	**NEW (D1).** Per item: confirm category + chemistry, weighed kg, condition, photos. Branches: li-ion → damage rubric; other → straight to price.
QR scan	`/job/[id]/scan`	Keep, **demoted to optional**. Manual entry is the primary path. "Generate QR" is deferred — it implies physical labelling we can't demo.
Damage rubric	`/job/[id]/items/[itemId]/damage`	Keep both states (clean + forced-Recycle). Li-ion items only. Visual 0.40 / Leakage 0.35 / Thermal 0.25 with photo slots — matches `DamageScores` exactly.
Computing	`/job/[id]/items/[itemId]/computing`	Keep. Six-layer stepper is honest — it reflects the real engine.

D. Quote

SCREEN	ROUTE	CHANGE
Verdict	`…/result`	Keep. Pathway hero, net value, P_min/P_rec/P_max band.

SCREEN	ROUTE	CHANGE
Breakdown	`…/result/breakdown`	Keep. Full revenue + cost lines. Agent-only — this is the deliberate inverse of the vendor rule.
Why	`…/result/why`	Keep rationale, alternatives, sensitivity, audit footer. **Cut the "AI explanation" button** — no `/api/explain` exists and it is not in scope.
HOLD	`…/result` (branch)	Keep. "Escalate to admin" must **do something**: flag the pickup + write a note, visible to admin later. Currently goes nowhere (W-small).
REVIEW	`…/result` (branch)	Keep.
Offer summary	`/job/[id]/offer`	**NEW.** The multi-item consequence: per-item prices roll up into **one** offer for the pickup. Wireframe had no such screen because it assumed one battery. This is what creates the `Offer` row and writes `offered`.

E. Collect & hand off

SCREEN	ROUTE	CHANGE
Confirm pickup	`/job/[id]/collect`	Keep. Photos, drop-off slot, contact confirm, signature, agent fee. Gated on `Offer.acceptedAt` (D7).
Vendor declines	`/job/[id]/collect` (branch)	Keep.
Receipt	`/job/[id]/receipt`	Keep. Writes `collected` + `PickupReceipt` + agent-fee `WalletTxn`.
Batch select	`/dropoff`	Keep — closest to the drawing.
Confirm hand-off	`/dropoff/confirm`	Keep. Hub, batch summary, GPS + timestamp, staff signature. Note: **agent-attested only** — there is no hub-staff app, so the receiving staff name is typed, not authenticated. Say so on screen.
Chain-of-custody receipt	`/dropoff/[batchId]`	Keep. New PDF template in `packages/pdf`.

F. Track, history, profile

SCREEN	ROUTE	CHANGE
My pickups	`/pickups`	Keep.
Lifecycle timeline	`/pickups/[id]`	Keep the screen, **replace the timeline** with `lifecycle-view.tsx` from `@clbipp/ui` (D5/W4).
Map	`/pickups/[id]/map`	Keep. Leaflet + OSM.
History	`/history`	Keep. **Rows must link to a real detail view** (currently self-linking).
Profile	`/profile`	Keep. Wallet, earnings, stats, offline queue, sign out. Add read-only **safety-training status** (D6). Remove "Cash out" and "Notifications" unless Batch 8 has room — dead buttons are worse than absent ones.

3. Schema delta — one migration, applied once

Same principle as Batch 0B: **one consolidated migration so nobody is blocked mid-week.** Everything else the agent app needs already exists.

```
ALTER  Pickup           + agentFeePaise        Int?      (D3)
ALTER  Offer            + acceptedAt           DateTime? (D7)
ALTER  BatteryItem      + damageVisual         Int?      (D1 — rubric, per item)
                        + damageLeakage        Int?
                        + damageThermal        Int?
                        + damageScore          Decimal?
                        + pathway              RecoveryPathway?
                        + traceId              String?   (links to PathwayDecision)
ALTER  enum WalletTxnKind + agent_fee                    (D3)
NEW    CustodyBatch     id, agentId, facilityId, batchNo, totalWeightKg,
                        itemCount, receivingStaffName, signatureUrl,
                        lat, lng, pdfUrl, handedOffAt
ALTER  Pickup           + custodyBatchId       String?   (D5 — derives
                                                          "pending drop-off")
```

Why `CustodyBatch` rather than reusing `DispatchManifest`: a manifest is facility → recycler (build doc §6). This is agent → facility. Different edge of the chain of custody, different actors, different document. Reusing it would make both wrong.

Not adding: any `PickupStatus` value (D5), any agent RLS policy set (D10), any admin-config table for fees (D3).

4. Batches and who owns them

Eleven batches across **three parallel lanes**. Each batch is one branch → one PR → merge, and each ends **green**: `npm run build` + `npm run test` + `npm run smoke`.

The lanes below are the `CLAUDE.md` ownership map, unchanged — this app happens to split along the same three seams the vendor app did, which is why the distribution needs no lane shift and no `LANE_OWNERSHIP.md` entry.

LANE	PERSON	OWNS HERE
A	**Aamir**	Auth + role gate, app shell + nav, job detail, safety checklist, tracking + realtime, history, profile, **and the cross-app seam** (D7). Identical to A's vendor-app lane.
B	**Khalid**	Schema + migration + seed, the decision engine and all pure pricing logic in `packages/core`, the PDF template, **and deploy**. All non-UI.
C	**Ali**	The on-site flow, screen by screen: intake → assessment → quote → collect → hub drop-off. The direct mirror of C's vendor-side request → offer → handover lane.

Batches

#	BATCH	OWNER	DELIVERABLE	EST.
0a	Schema + seed	**B**	The one migration (§3) — fully specified, no design work left. Seed: agent + assigned pickups at every relevant stage, `MarketPrices` row, `Facility` row.	0.4d

#	BATCH	OWNER	DELIVERABLE	EST.
0b	App scaffold	**A**	`apps/agent` shell, tokens, `(agent)` layout, bottom nav, `proxy.ts` with `allowRoles: ['agent']` (**must live at `src/proxy.ts`** — an unregistered guard fails OPEN). Agent app added to `npm run smoke`.	0.5d
1	Day view + job detail	**A**	Login, home (assigned jobs, stats, offline banner), job detail with Maps deep link + `tel:`, **Arrived →** writes `arrived`. First service-role agent write — sets the pattern for every batch after.	0.75d
2	Safety checklist	**A**	The missing mandatory gate (W1). Chemistry-aware items, writes `SafetyChecklist`, blocks intake until `passed`. Small, high-visibility — this is what HR looks for first.	0.5d
3	Multi-item intake	**C**	Item list + per-item confirm: category, chemistry, weighed kg, condition, photos via service-role upload. QR scan as optional entry. **The biggest UI batch — the D1 cost lands here.**	1.0d
4	Engine + pricing	**B**	Fix both D9 defects **in the engine**, with tests. `MarketData` adapter, `rupeesToPaise`, agent-fee rule (D3), `POST /api/quote`, persist `PathwayDecision`. Non-li-ion priced through the existing `estimateQuote`. All unit-tested in packages — no test lives in an app.	1.2d
5a	Quote screens	**C**	Verdict / Breakdown / Why, HOLD + REVIEW branches, roll-up offer screen → creates `Offer`, writes `offered`. Built against a mock `QuoteOutput` until 4 lands (see seams below).	0.75d
5b	Cross-app seam	**A**	The D7 change to the **customer** app: `acceptOffer` sets `Offer.acceptedAt` and stops writing `collected`; `/handover` becomes a POST. Small, but it touches a deployed app — A's seam, as always.	0.4d
6	Collect	**C**	Photos, signature capture, receipt, writes `collected` + `PickupReceipt` + agent-fee `WalletTxn`. Vendor-declines branch.	0.75d
7a	Hub drop-off UI	**C**	Batch select, confirm with GPS + staff signature, writes `CustodyBatch`.	0.6d
7b	Custody PDF	**B**	Chain-of-custody receipt template in `packages/pdf`, alongside the three existing ones. Pure render, no UI.	0.4d
8	Track, history, profile	**A**	`/pickups` + lifecycle via `lifecycle-view`, Leaflet map, history with a real detail view, profile + wallet + earnings + training status. Agent-scoped `status_events` SELECT policy for realtime (D10).	0.75d
9	Deploy + verification	**B**	Vercel project for `apps/agent`, env, redirect URLs. Then a full pass across batch seams, all three present, both apps side by side.	0.5d

Per person: A ≈ 2.9d · B ≈ 2.5d · C ≈ 3.1d, across seven calendar days. That is deliberately under 7 — the slack is integration, review and the things that always appear.

The four seams — agree these on day 1, then nobody blocks

Everything that could make one person wait on another is a **typed contract that already exists or is one field wide**. Fix them in the day-1 kickoff and the three lanes never touch each other again until Batch 9.

SEAM	CONTRACT	WHO WAITS ON WHOM
Engine → quote screens	**`QuoteOutput`** in `decision-engine/src/decisionEngine/types.ts` — already fully typed today	**Nobody.** C builds 5a against a mock `QuoteOutput` in `packages/core/src/mock-data.ts`, the repo's standard stub-data pattern. Swapping to the real `POST /api/quote` is a one-line import change.

SEAM	CONTRACT	WHO WAITS ON WHOM
Vendor acceptance → collect	`Offer.acceptedAt` (one nullable column)	**Nobody.** A writes it (5b), C reads it (6). It's in B's day-1 migration, so it exists before either starts.
Agent writes → everything	The service-role server-action pattern A establishes in Batch 1	C copies the pattern from Batch 1's PR. One read, no conversation.
Screens → nav	The route table in §2	A's Batch 0b ships the shell with every route stubbed, so links resolve before the screens behind them exist.

The only genuine ordering constraint is day 1: B's migration and A's shell both need to land before C can start Batch 3. Both are fully specified here and carry no open design questions, which is why they are first and why they are small.

Day by day

DAY	A — AAMIR	B — KHALID	C — ALI
1	0b scaffold + auth gate	**0a migration + seed (first thing)**	Reads §2/§3, preps components
2	1 day view + job detail	4 engine defects + tests	3 intake (vs. mocks)
3	2 safety checklist	4 market adapter + fee rule	3 intake + damage rubric
4	5b cross-app seam	4 `/api/quote` + persist	5a quote screens (vs. mock)
5	8 tracking + realtime	7b custody PDF	5a → real API · 6 collect
6	8 history + profile	9 deploy prep	6 collect · 7a drop-off
7	**Integration + verification — all three, together**		

5. Risks — what actually blows the week

1. **The schedule has no slack.** 7.5 days of work in 7 days. If anything slips, something must be cut, and it should be a **decided** cut rather than a half-finished batch. Cut order: **Batch 8 extras** (map, then history detail, then wallet cash-out) → **QR scan** (Batch 3) → **HOLD/REVIEW branches** (Batch 5). Batches 0–7 minus those are the demo path and cannot be cut.
2. **Batch 3 is the one that will overrun.** Multi-item intake is the most new UI in the plan and the direct cost of D1. If it runs long by more than half a day, fall back to **one item per pickup for li-ion** and keep the loop only for the simple path — a contained retreat, decided on the spot, not a rewrite.
3. **The cross-app seam (D7) is the highest-risk *correctness* item**, **not the highest-effort one. It changes a merged, deployed customer app. Do it in Batch 5 as specified —** not** opportunistically in an earlier batch, and not at the end when there's no time to fix what it breaks.
4. **The engine is now editable, which cuts both ways (D0).** Fixing the two real defects in it is right. Refactoring it because it could be nicer is not — it is 1,476 lines with 20 tests and a live pricing surface, and there is no week to spend re-earning that. The bar: **fix defects and anything the HR documents contradict; leave working economics alone.** Any change that moves a price says so in its PR.
5. **Deploy is not free.** Batch 12 of the revamp is still in flight with Khalid. A second Vercel project, its own env vars and its own redirect URLs are half a day, and it is the last thing anyone wants to discover on day 7.

6. Lanes — settled

All three of us are available for this app, so the 2026-08-09 override (A covering all three lanes for the customer-app revamp) **lapses here**. Ownership reverts to the `CLAUDE.md` map, and the split in §4 is that map applied to this app — A on auth/shell/tracking/seam, B on schema + logic + deploy, C on the on-site flow.

No lane shift is needed and no `LANE_OWNERSHIP.md` entry is required. That is not a coincidence: the agent app decomposes along the same three seams the vendor app did, because it is the same architecture seen from the other side.

Two standing rules from `CLAUDE.md` that matter more than usual this week, given three people moving in parallel on a seven-day budget:

- **Don't edit another lane's area, even if it's faster.** If you need something that isn't built yet, stub it against the agreed shape (see the seam table in §4) and leave a `// TODO: swap for real <X> once <owner> ships it`.
- **One feature = one small branch/PR.** Three people and a shared `main` is the configuration where a bundled PR costs everyone a day.

---

7. Open questions — not blocking, worth asking

These go to the company alongside the seven still unanswered from `COMPANY_FLOW_REVIEW_2026-08-07.md`. **None of them block the build**; each has a stated assumption above.

1. Is the agent an independent partner paid per job, or salaried staff? (D3 assumes partner — it's the wireframe's own reading and matches the EpiCircle model HR cites, but no HR document actually says it.)
2. Who assigns jobs — an ops person, or an automatic zone rule? (D2 assumes pushed; the assigner itself is the admin app's problem, not ours.)
3. Is there a hub-staff app? Our drop-off is agent-attested only, which is a real weakness in a chain-of-custody document.
4. Does the agent need to see the vendor's declared photos before arriving? (We show them — it seems obviously useful, but it is our call, not theirs.)
5. Is "Entroview" a system we are eventually expected to integrate, or just the name in the engine spec? The wireframe puts it on screen as a v1 limitation.